

How the MLP Works — The Complete Workflow

Read this whenever you feel lost. It explains the entire journey from input to output.

The Three Classes — What Each One Is

Class	What it is	Analogy
Neuron	One calculation unit	One interviewer scoring a candidate
Layer	A row of neurons	A panel of interviewers
MLP	Multiple layers stacked	Multiple interview rounds

The Journey of an Input

Let's say you build this network:

```
n = MLP(3, [4, 4, 1])
x = [2.0, 3.0, -1.0]
output = n(x)
```

3 numbers go in. 1 number comes out. Here is exactly what happens in between.

Step 1 — MLP receives the input

```
# You call n(x) — this triggers MLP.__call__(x)

def __call__(self, x):
    for layer in self.layers:
        x = layer(x)      # pass x through each layer one by one
    return x              # final x is the answer
```

MLP has 3 layers. It loops through them one by one. The output of each layer becomes the input of the next. MLP itself does no maths — it just passes x along.

Step 2 — MLP calls Layer 1

```
x = layer(x)
# This triggers Layer.__call__(x)
```

x = [2.0, 3.0, -1.0] enters Layer 1. Layer 1 has 4 neurons.

```
def __call__(self, x):
    outs = [n(x) for n in self.neurons]  # call every neuron with same x
    return outs[0] if len(outs) == 1 else outs
```

Layer loops through its 4 neurons. For each neuron it calls `n(x)` — this triggers `Neuron.__call__(x)`.

Step 3 — Layer calls Neuron (happens 4 times)

Each of the 4 neurons receives the full input `[2.0, 3.0, -1.0]`. Same input. But each neuron has its OWN weights — so each produces a different output.

```
def __call__(self, x):
    act = sum((wi*xi for wi, xi in zip(self.w, x)), self.b)
    out = act.tanh()
    return out
```

What each neuron does:

- Pairs its weights with the inputs: $(w_0, 2.0)$, $(w_1, 3.0)$, $(w_2, -1.0)$
- Multiplies each pair: $w_0 \times 2.0$, $w_1 \times 3.0$, $w_2 \times -1.0$
- Adds them all up plus bias: $w_0 \times 2.0 + w_1 \times 3.0 + w_2 \times -1.0 + b$
- Squashes with `tanh` → output between -1 and +1
- Returns one single number

```
N1 → 0.71
N2 → -0.42
N3 → 0.13
N4 → 0.95
```

Step 4 — Control returns to Layer

All 4 neurons have returned their values. Layer collects them into a list:

```
outs = [0.71, -0.42, 0.13, 0.95]
```

Layer 1 has 4 neurons so it returns the full list:

```
return outs    # [0.71, -0.42, 0.13, 0.95]
```

Step 5 — Control returns to MLP

MLP receives `[0.71, -0.42, 0.13, 0.95]` back from Layer 1. It updates `x`:

```
x = [0.71, -0.42, 0.13, 0.95]
```

Now `x` is 4 numbers instead of 3. MLP moves to the next iteration of its loop — Layer 2.

Step 6 — Same thing happens in Layer 2

$x = [0.71, -0.42, 0.13, 0.95]$ enters Layer 2. Layer 2 also has 4 neurons. Each neuron receives all 4 numbers. Each produces one output. Layer 2 returns $[0.33, 0.87, -0.21, 0.64]$.

Step 7 — Same thing happens in Layer 3

$x = [0.33, 0.87, -0.21, 0.64]$ enters Layer 3. Layer 3 has only 1 neuron — the output neuron. That neuron receives all 4 numbers. Produces one output: 0.54

Layer 3 has only 1 neuron so it returns the value directly, not a list:

```
return outs[0]    # 0.54    not [0.54]
```

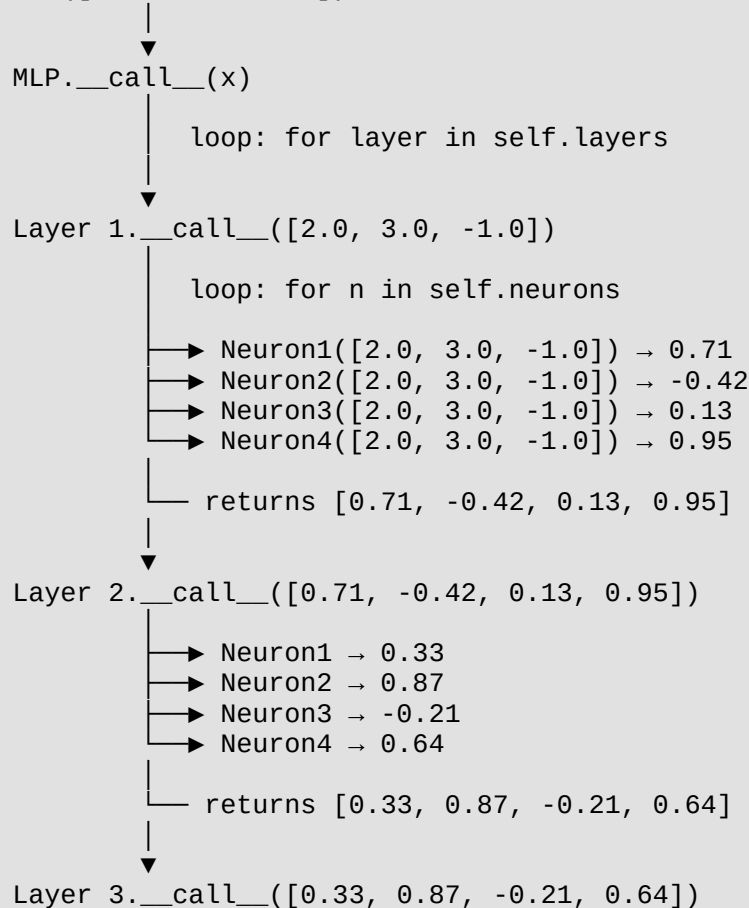
Step 8 — MLP returns the final answer

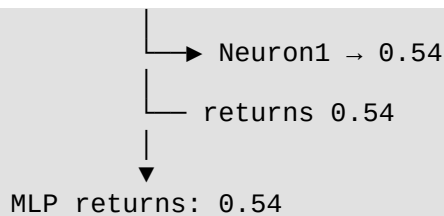
```
return x    # 0.54
```

That's the network's prediction.

The Complete Picture

You call: `n([2.0, 3.0, -1.0])`





Why Each Class Exists

Neuron exists because the maths lives here. Every multiplication, every addition, every tanh — happens inside Neuron. Everything else just moves data around.

Layer exists because we need multiple neurons running in parallel. It is the organiser — collects neurons, feeds them all the same input, gathers their outputs.

MLP exists because we need multiple layers in sequence. It is the conductor — passes x through each layer, letting the representation transform at each step.

The Key Insight You Had

A neural network is just nested loops.

```
for layer in layers:           # MLP loops through layers
    for neuron in layer:        # Layer loops through neurons
        for weight in neuron:   # Neuron loops through weights
            output += weight * input
```

Karpathy wraps this in classes so it looks elegant. But underneath — it is loops inside loops inside loops.

The magic is not the loops. The magic is that the weights learn. Everything else is just loops.

What Happens Next — The Training Loop

Right now the network gives a random answer — 0.54. The correct answer might be 1.0. The network was wrong.

The training loop fixes this:

1. Forward pass → network makes a prediction (0.54)
2. Compute loss → how wrong was it? ($\text{loss} = (0.54 - 1.0)^2 = 0.21$)
3. `loss.backward()` → compute gradient for every weight in every neuron
4. Update weights → nudge every weight slightly in the right direction
5. Zero gradients → reset for next round
6. Repeat → prediction gets closer to correct each time